

```
In [1]: # Importing packages
import pandas as pd
import re
import sys
import os

BASE = r"C:\meghanath\1. UNCC\Fall 2025\SECURITY ANALYTICS\Project-5 DNS Log Analys

# Adding freq-master to Python path
sys.path.insert(1, os.path.join(BASE, "freq-master"))

from freq import FreqCounter
```

```
In [2]: # Loading DNS Log file
dns_data = pd.read_json(os.path.join(BASE, "dnslog.json"), lines=True)
dns_data.head()
```

```
Out[2]:
```

	Unnamed: 0	ts	id.orig_h	id.orig_p	id.resp_h	id.resp_p	proto	
0	16583	2024-04-07 12:00:00	10.0.1.11	55555	10.0.0.10	53	udp	9.a.6.e.f.9.c.c.a.8.5.e.!
1	16585	2024-04-07 12:00:00	10.0.1.11	47742	10.0.0.10	53	udp	1.0
2	16586	2024-04-07 12:00:00	10.0.1.11	39923	10.0.0.10	53	udp	2.0.0.0.1.0.0.0.0.0.0.0
3	16587	2024-04-07 12:00:00	10.0.1.11	57318	10.0.0.10	53	udp	!
4	16584	2024-04-07 12:00:00	10.0.1.11	54568	10.0.0.10	53	udp	2.0

```
In [3]: # Initializing and loading frequency counter
domain_frequency_counter = FreqCounter()
domain_frequency_counter.load(os.path.join(BASE, "freq-master", "freqtable2018.freq
```

```
In [11]: # Function to extract domains from URLs
def root_domains(queries):
    extracted_domains = []
    for query in queries:
        # Clean and extract after optional 'www.'
        match = re.search(r"(?:www\.)?(.*)", str(query))
        if match:
            extracted_domains.append(match.group(1).strip("/"))
```

```

        else:
            extracted_domains.append(query)
    return extracted_domains

```

```

In [13]: # Extracting domains from the 'query' field in DNS Log
queries_list = dns_data["query"]
extracted_domains = root_domains(queries_list)

print(f"Total domain count: {len(extracted_domains)}")

```

Total domain count: 29368

```

In [7]: # Function to calculate the frequency for each domain
def domain_frequency(domain_list):
    frequency_list = []
    for domain in domain_list:
        try:
            val = domain_frequency_counter.probability(str(domain))
            freq_val = float(val[0]) if isinstance(val, (list, tuple)) else float(v
        except Exception:
            freq_val = float("nan")
        frequency_list.append(freq_val)
    return domain_list, frequency_list

```

```

In [8]: # Computing unique domains and their frequencies
unique_domains, domain_frequencies = domain_frequency(extracted_domains)

# Check
print("Total domains scored:", len(domain_frequencies))
pd.DataFrame({"domain": unique_domains, "frequency": domain_frequencies}).head()

```

Total domains scored: 29368

```

Out[8]:

```

	domain	frequency
0	9.a.6.ef.9.c.c.a.8.5.e.5.b.5.8.0.0.0.0.0.0....	1.6578
1	1.0.16.172.in-addr.arpa	4.4872
2	2.0.0.0.1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0....	1.9804
3	9.0.0.10.in-addr.arpa	4.5263
4	2.0.16.172.in-addr.arpa	4.6108

```

In [9]: # Adding the new unique domains and frequencies to the dataset
dns_data["domain"] = unique_domains
dns_data["frequency"] = domain_frequencies
dns_data.head()

```

Out[9]:	Unnamed: 0	ts	id.orig_h	id.orig_p	id.resp_h	id.resp_p	proto	
	0	2024-04-07 12:00:00	10.0.1.11	55555	10.0.0.10	53	udp	9.a.6.e.f.9.c.c.a.8.5.e.!
	1	2024-04-07 12:00:00	10.0.1.11	47742	10.0.0.10	53	udp	1.0
	2	2024-04-07 12:00:00	10.0.1.11	39923	10.0.0.10	53	udp	2.0.0.0.1.0.0.0.0.0.0.0
	3	2024-04-07 12:00:00	10.0.1.11	57318	10.0.0.10	53	udp	!
	4	2024-04-07 12:00:00	10.0.1.11	54568	10.0.0.10	53	udp	2.0

```
In [10]: # Loading the Top 1M List
top1m_df = pd.read_csv(os.path.join(BASE, "top-1m.csv"), header=None, names=["rank"
```

```
In [12]: top1m_domains_list = root_domains(top1m_df["domain"])
top1m_set = set(str(d).strip().lower() for d in top1m_domains_list)
len(top1m_set)
```

Out[12]: 891544

```
In [14]: dns_data["Present_in_top1M"] = (
    dns_data["domain"].astype(str).str.strip().str.lower().isin(top1m_set)
)

# Check
dns_data.loc[:, ["domain", "frequency", "Present_in_top1M"]].head(10)
```

Out[14]:

	domain	frequency	Present_in_top1M
0	9.a.6.e.f.9.c.c.a.8.5.e.5.b.5.8.0.0.0.0.0.0.0....	1.6578	False
1	1.0.16.172.in-addr.arpa	4.4872	False
2	2.0.0.0.1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0....	1.9804	False
3	9.0.0.10.in-addr.arpa	4.5263	False
4	2.0.16.172.in-addr.arpa	4.6108	False
5	amazon.com	8.0248	True
6	google.com	6.6009	True
7	YOLP.com	6.2303	False
8	twitter.com	7.9837	True
9	uncc.edu	4.0202	True

In [15]: *# Checking if each domain is part of the Top 1M List*

```
top_1M_check = ["No"] * len(dns_data["domain"])
for index, domain in enumerate(dns_data["domain"]):
    d = str(domain).strip().lower()
    if d and d in top1m_set:
        top_1M_check[index] = "Yes"

dns_data["Present_in_top1M"] = top_1M_check
```

In [16]: *# Keeping the required columns in a clean view*

```
final_dns_data = dns_data[["id.orig_h", "domain", "query", "frequency", "Present_in_top1M"]]
final_dns_data = final_dns_data.rename(columns={"id.orig_h": "ip"})

# quick peek
final_dns_data.head()
```

Out[16]:

	ip	domain	query	fr
0	10.0.1.11	9.a.6.e.f.9.c.c.a.8.5.e.5.b.5.8.0.0.0.0.0.0.0....	9.a.6.e.f.9.c.c.a.8.5.e.5.b.5.8.0.0.0.0.0.0.0....	
1	10.0.1.11	1.0.16.172.in-addr.arpa	1.0.16.172.in-addr.arpa	
2	10.0.1.11	2.0.0.0.1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0....	2.0.0.0.1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0....	
3	10.0.1.11	9.0.0.10.in-addr.arpa	9.0.0.10.in-addr.arpa	
4	10.0.1.11	2.0.16.172.in-addr.arpa	2.0.16.172.in-addr.arpa	

In [17]: **import** os

```
OUT_JSON = os.path.join(BASE, "enriched_dnslog.json")
final_dns_data.to_json(OUT_JSON, orient="records")
print("Saved:", OUT_JSON, "| rows:", len(final_dns_data))
```

Saved: C:\meghanath\1. UNCC\Fall 2025\SECURITY ANALYTICS\Project-5 DNS Log Analysis
\enriched_dnslog.json | rows: 29368

```
In [18]: # Creating a copy so the original dns_data stays untouched
new_dns_data = dns_data.copy()
```

```
In [24]: ips = sorted({
    str(ip)
    for ans in new_dns_data["answers"].dropna()
    for ip in (ans if isinstance(ans, list) else [ans])
})

# One entry per domain (deduped); keep the most frequent mapping per IP
mapped_domain_results = []
seen = set()

def _norm(d): # collapse www. duplicates, case-insensitive
    d = str(d).strip().lower()
    return d[4:] if d.startswith("www.") else d

for ip in ips:
    pairs = mapped_ip(ip, new_dns_data) # [(domain, count), ...] or []
    if not pairs:
        continue
    dom, cnt = pairs[0]
    key = _norm(dom)
    if key not in seen:
        mapped_domain_results.append([dom, cnt])
        seen.add(key)

# sort by count (desc) for a cleaner list
mapped_domain_results.sort(key=lambda x: x[1], reverse=True)
```

```
In [25]: for row in mapped_domain_results:
    print(row)
```

```
['sms.us-east-1.amazonaws.com', 3157]
['www.YOLP.com', 2000]
['www.wikipedia.org', 1717]
['www.fb.com', 1693]
['www.uncc.edu', 1689]
['www.google.com', 1661]
['www.amazon.com', 1657]
['www.twitter.com', 1583]
['loggingest.test.int', 1324]
['vcenter.test.int', 614]
['dc01.test.int', 226]
['dc02.test.int', 145]
['s3.amazonaws.com', 143]
['sns.us-east-1.amazonaws.com', 132]
['connector-platform-upgrade-info.s3-us-west-2.amazonaws.com', 119]
['www.mompie.com', 100]
['223954885._teamviewer._tcp.local', 63]
['2.0.16.172.in-addr.arpa', 57]
['10.0.0.10.in-addr.arpa', 47]
['nessus.test.int', 39]
['www.Y0LP.com', 30]
['241.0.0.10.in-addr.arpa', 30]
['9.0.0.10.in-addr.arpa', 30]
['cuckoo.test.int', 25]
['11.1.0.10.in-addr.arpa', 24]
['500111316._teamviewer._tcp.local', 23]
['statsfe2.update.microsoft.com', 22]
['4.1.0.10.in-addr.arpa', 21]
['sms-b-us-east-1-kki7yqjnygi5vodvboyrijbjkybo6dbf.s3.amazonaws.com', 10]
['awsconnector.us-east-1.amazonaws.com', 6]
['_ldap._tcp.default-first-site-name._sites.dc._msdcs.test.int', 6]
['201.1.0.10.in-addr.arpa', 6]
['sls.update.microsoft.com', 6]
['www.CH183317.net', 5]
['hp900bfc.local', 5]
['sqm.telemetry.microsoft.com', 5]
['tools.google.com', 5]
['251.0.0.10.in-addr.arpa', 5]
['_kerberos._tcp.default-first-site-name._sites.dc._msdcs.test.int', 4]
['11.0.0.10.in-addr.arpa', 4]
['_vlmcs._tcp.test.int', 4]
['RT4x78.net', 3]
['rules.emergingthreatspro.com', 3]
['_googlecast._tcp.local', 3]
['ppa.launchpad.net', 3]
['telecommand.telemetry.microsoft.com', 3]
['_ldap._tcp.default-first-site-name._sites.test.int', 3]
['4.53.231.41.in-addr.arpa', 3]
['3.1.0.10.in-addr.arpa', 3]
['4.89.75.208.in-addr.arpa', 3]
['lab-linux.local', 2]
['autoupdate.geo.opera.com', 2]
['us.archive.ubuntu.com', 2]
['ec2.amazonaws.com', 2]
['dl.opera.com', 2]
['photosmart c7100 series [900bfd]._printer._tcp.local', 2]
```

```

['ctld1.windowsupdate.com', 2]
['dc.applicationinsights.microsoft.com', 2]
['_afpovertcp_tcp.local', 2]
['tele.trafficmanager.net', 2]
['esx04.test.int', 1]
['lab-linux-2.local', 1]
['www.snort.org', 1]
['google.com', 1]
['pool.ntp.org', 1]
['security.ubuntu.com', 1]
['iam.amazonaws.com', 1]
['1.64.109.50.in-addr.arpa', 1]
['cit011pt.local', 1]
['lightforgetouch.local', 1]
['_raop_tcp.local', 1]
['photosmart c7100 series [900bfd]_pdl-datastream_tcp.local', 1]
['57.151.42.74.in-addr.arpa', 1]
['142.4.40.74.in-addr.arpa', 1]
['34.3.40.74.in-addr.arpa', 1]
['clients2.google.com', 1]
['98.119.223.206.in-addr.arpa', 1]
['49.0.229.216.in-addr.arpa', 1]
['fe2.update.microsoft.com', 1]
['_ipp_tcp.local', 1]
['50.5.40.67.in-addr.arpa', 1]
['download3.operacdn.com', 1]
['238.216.58.216.in-addr.arpa', 1]
['_pdl-datastream_tcp.local', 1]
['1.0.0.10.in-addr.arpa', 1]
['aws-connector-config.s3.amazonaws.com', 1]
['server-migration-service-upgrade.s3-us-west-2.amazonaws.com', 1]
['163.155.22.50.in-addr.arpa', 1]
['vuxml.freebsd.org', 1]

```

In [26]: *# Checking for naked IP or not (using mapped_ip)*
test_ips = ["137.245.120.50", "104.244.42.1", "172.20.56.90", "99.84.181.52"]

```

for ip in test_ips:
    pairs = mapped_ip(ip, new_dns_data)  # [(domain, count), ...] or []
    if not pairs:
        print(f"{ip} : NAKED IP")
    else:
        domain, cnt = pairs[0]           # top mapping
        print(f"{ip} : MAPPED -> {domain} ({cnt})")

```

```

137.245.120.50 : NAKED IP
104.244.42.1 : MAPPED -> www.twitter.com (1583)
172.20.56.90 : NAKED IP
99.84.181.52 : NAKED IP

```

In [27]: *# Checking high-risk domains for suspicious activity*
suspicious_domains = ["zak0n", "mitula", "ae0333a7", "M0M3367"]

```

for d in suspicious_domains:
    r = domain_frequency_counter.probability(d)

```

```
prob = r[0] if isinstance(r, (list, tuple)) else r
print(f"Probability of '{d}': {prob}")
```

```
Probability of 'zak0n': 2.0316
Probability of 'mitula': 8.3011
Probability of 'ae0333a7': 1.4058
Probability of 'M0M3367': 1.7917
```

In []: